ChatGPT

Roadmap hai tuần buổi tối để hoàn thiện robot social Mone

Executive summary

Dự án Mone (robot social) có rủi ro lớn nhất không còn nằm ở "train model", mà nằm ở **tích hợp hệ thống**: mic ổn định (đầu vào), chạy model trên Jetson Nano (compute), persona & prompt (tương tác), điều khiển động cơ qua ESP32 + H-bridge (hành vi), cấp nguồn pin độc lập (tính cơ động), quản lý nhiệt (độ bền), và **tối ưu latency end-to-end < 2 giây** (trải nghiệm). Các hạng mục phần cứng/Jetson trong báo cáo này bám theo khả năng/giới hạn chính thức của Jetson Nano Developer Kit: nguồn **Micro-USB 5V⎓2A** (mặc định), nguồn qua **jack DC J25 5V⎓4A**, lựa chọn nguồn bằng **jumper J48**, và có thể cấp **5V⎓5A qua header J41 (2.5A mỗi pin)**; điện áp tối thiểu để module hoạt động là **4.75V**. [1]

Do môi trường chat này hiện **không truy cập được Google Sheet bạn gửi** (không có kết nối nguồn/connector), roadmap dưới đây được xây từ bộ hạng mục bạn đã mô tả từ sheet (mic test, deploy Jetson Nano, persona/prompt, ESP32 motor control, chassis assembly, pin độc lập, tản nhiệt, latency <2s). Nếu bạn export sheet sang PDF/CSV và upload trực tiếp vào chat, mình có thể "khóa" lại mức độ chi tiết theo đúng tên task/owner/deadline trong sheet.

Chiến lược tổng thể (tối ưu cho 3–4 giờ mỗi tối):
Trong 4 buổi tối đầu, ưu tiên "đường ống" tối thiểu chạy được trên Jetson: **record → preprocess → inference → phản hồi** + đo latency và nhiệt. Jetson Nano được khuyến nghị chuẩn bị đúng nguồn (đặc biệt nếu kéo thêm USB/motor driver), vì NVIDIA nêu rõ nếu tải vượt 2A (do ngoại vi) thì cần chuyển sang cấp 5V⎓4A qua J25 (bằng J48) hoặc cấp qua J41. [1]
Tuần thứ hai, "cắm" hành vi vận động (ESP32 PWM) và nguồn pin độc lập, sau đó tối ưu latency và tính ổn định bằng công cụ đo/giám sát sẵn có như **tegrastats** (tiện ích báo cáo CPU/GPU/memory, có sẵn trên Jetson). [2]

Định nghĩa "xong để demo" sau 14 buổi tối (đề xuất):
Mone nghe được mic ổn định, suy luận cảm xúc tối thiểu 2 lớp (ví dụ vui/buồn), phản hồi theo persona (rule/prompt), di chuyển được các lệnh cơ bản, chạy pin độc lập tối thiểu 20–30 phút, không quá nhiệt/throttle rõ rệt, và latency P95 tiến gần hoặc đạt <2 giây theo phép đo thống nhất.

Bối cảnh, giả định, và tiêu chí "xong để demo"

Ràng buộc thời gian: bạn chỉ rảnh **buổi tối**, mỗi tối giả định **3–4 giờ**, không chốt giờ bắt đầu/kết thúc. Roadmap vì vậy ưu tiên các nhiệm vụ có thể "ra kết quả" trong một phiên ngắn, và đẩy các việc tốn thời gian treo máy (in 3D lâu, hàn/cơ khí dài, sạc/đóng pack pin…) sang backlog ban ngày.

Giả định kỹ thuật (vì sheet không truy cập trực tiếp được trong chat): - Compute: Jetson Nano Developer Kit chạy JetPack/Linux; JetPack cung cấp desktop Linux + driver + thư viện AI/CV theo mô tả của tài liệu kit. [3]
- Audio: mic hiện đang được test (ví dụ vui/buồn). Bạn có thể đang có `test_mic.py` hoặc script tương tự. - Motor: động cơ DC + mạch cầu H (H-bridge) và điều khiển bằng ESP32 (C++/ESP-IDF hoặc Arduino). ESP32 có thể phát PWM bằng **LEDC** (khuyến nghị khi cần đơn giản) hoặc dùng **MCPWM** (đúng bài cho motor control, có ví dụ brushed DC + H-bridge trong tài liệu). [4]
- Nguồn: Jetson Nano có thể cần chuyển sang cấp qua J25 5V⎓4A hoặc J41 5V⎓5A khi gắn ngoại vi; NVIDIA ghi rõ J25 và J48, cũng như yêu cầu tối thiểu 4.75V tại module. [1]
- Nhiệt: tản nhiệt thụ động trên kit được NVIDIA ghi "support 10W ở 25°C ambient", và carrier board có **fan header J15 4-pin, hỗ trợ PWM và tach**. [5]

Tiêu chí nghiệm thu cốt lõi (khuyến nghị đo bằng số, không "cảm giác"): - **Mic ổn định:** cùng 1 câu nói/ lưu lượng môi trường, dự đoán lặp lại không "nhảy" vô cớ; không clipping; sample rate & format đồng nhất. - **Inference chạy trên Jetson:** 1 lệnh chạy end-to-end, không cần sửa tay, có log. - **Latency:** đo theo pipeline thống nhất (record + preprocess + inference + response + gửi lệnh motor nếu có). Mục tiêu cuối: <2s là mục tiêu trải nghiệm; tối thiểu phải có biểu đồ/CSV đo P50/P95. - **Motor control:** 5 lệnh cơ bản (forward/back/left/right/stop) chạy ổn định, có "failsafe stop". - **Nguồn pin độc lập:** robot chạy không dây, không brownout/reset khi motor tăng tải. - **Thermal:** theo dõi tegrastats; không throttle đột ngột, có phương án quạt J15.

Kiến trúc tích hợp và các quyết định công nghệ quan trọng

Kiến trúc tham chiếu tối giản

```
flowchart TD
  U[Người dùng nói] --> MIC[Mic/Audio interface]
  MIC --> CAP[Capture (ALSA/arecord/PortAudio)]
  CAP --> FEAT[Preprocess + Feature Extract (MFCC/log-mel...)]
  FEAT --> EMO[Emotion Model Inference trên Jetson]
  EMO --> DM[Dialogue/Policy: persona + prompt + rules]
  DM --> OUT[Output: text/voice/UI]
  DM --> ACT[Behavior mapper]
  ACT -->|UART/Wi-Fi| ESP[ESP32]
  ESP --> HBR[H-bridge]
  HBR --> MOT[DC motors + chassis]
  EMO --> MET[Logging: latency + confidence]
  CAP --> MET
  ESP --> MET
  JET[Jetson tegrastats/power] --> MET
```

Quyết định quan trọng cần "khóa" sớm trong 2–3 buổi tối đầu

Chọn đường deploy model lên Jetson theo "2 lớp":
Lớp 1 (để chạy sớm): chạy bằng runtime hiện bạn đang dùng (thường là TensorFlow/PyTorch CPU/GPU).

Lớp 2 (để đạt <2s): tối ưu bằng TensorRT hoặc ONNX Runtime + TensorRT EP khi phù hợp, vì TensorRT là SDK tối ưu inference hiệu năng cao cho GPU NVIDIA, hỗ trợ nhiều tối ưu như mixed precision và tối ưu hoá runtime engine. [6]

Chọn stack audio ổn định trên Linux/Jetson:
Để "đi nhanh", dùng **arecord/aplay** (ALSA) để kiểm tra thiết bị và ghi âm chuẩn hóa; arecord là trình ghi âm dòng lệnh cho ALSA, hỗ trợ nhiều soundcards/devices. [7]
Nếu dùng PulseAudio, cần ý thức về quản lý latency (buffering); PulseAudio có tài liệu riêng về "latency control". [8]

Chọn PWM motor control trên ESP32:
- **LEDC**: tài liệu ESP-IDF nêu LEDC chủ yếu cho LED nhưng **có thể dùng để tạo PWM cho mục đích khác**, có nhiều kênh PWM độc lập. [9]
- **MCPWM**: được thiết kế cho motor control/power electronics; tài liệu có phần hướng dẫn cấu hình điều khiển **brushed DC motor dùng H-bridge**. [10]

Nguồn và nhiễu motor:
Jetson Nano kit mặc định dùng Micro-USB 5V⎓2A; nếu tổng tải >2A (USB ngoại vi, v.v.) NVIDIA khuyến nghị chuyển qua jack DC J25 5V⎓4A bằng cách gắn jumper J48, hoặc cấp 5V⎓5A qua header J41; đồng thời module cần tối thiểu 4.75V. [1]
Điều này ảnh hưởng trực tiếp đến thiết kế pin: bạn cần rail 5V "sạch"/ổn định cho Jetson, và rail motor tách hoặc ít nhất có lọc tốt.

Bảng so sánh lựa chọn

Chiến lược deploy inference trên Jetson

Lựa chọn	Mô tả	Ưu điểm	Nhược điểm	Khi nên chọn	Bằng chứng nguồn chính
TensorRT (native)	Dùng TensorRT để build engine tối ưu và chạy inference	TensorRT là SDK tối ưu và tăng tốc inference trên GPU NVIDIA, mục tiêu low-latency/ high-throughput; hỗ trợ mixed precision (FP16/INT8…) [6]	Tốn công convert/build engine; rủi ro op không hỗ trợ; cần benchmark & validate	Khi latency là KPI số 1 và model tương đối "chuẩn" (CNN/MLP/ RNN đơn giản)	TensorRT docs mô tả mục tiêu tối ưu inference và chạy engine [11]

Lựa chọn	Mô tả	Ưu điểm	Nhược điểm	Khi nên chọn	Bằng chứng nguồn chính
ONNX Runtime (ORT) + TensorRT EP	Chạy model ONNX trên ORT và bật TensorRT Execution Provider	ORT có execution providers; TensorRT EP tận dụng TensorRT để tăng hiệu năng trên GPU NVIDIA [12]; có hướng dẫn Python và khuyến nghị đăng ký thêm CUDA EP để fallback khi TensorRT không hỗ trợ node [13]	Phát sinh phức tạp EP/ fallback; phải đảm bảo build/wheel tương thích CUDA/ TensorRT	Khi bạn muốn API Python "êm", dễ tích hợp, nhưng vẫn muốn hưởng lợi TensorRT	ORT TensorRT EP docs và mô tả EP model [14]
TensorFlow Lite (TFLite)	Convert sang .tflite và chạy bằng TFLite Interpreter	Interpreter API rõ ràng, cơ chế allocate_tensors trước khi infer [15]; có Delegate API để bridge sang accelerator [16]	GPU delegate tài liệu chính thống tập trung Android/iOS (không phải Jetson/Linux) [17]; hiệu năng trên Jetson có thể không tốt bằng TensorRT; convert có thể khó tùy model	Khi model cực nhỏ, muốn footprint nhẹ, hoặc bạn đã có pipeline TFLite sẵn	Delegate/ Interpreter docs [18]

Khuyến nghị thực dụng cho 2 tuần buổi tối:
- Buổi tối 1–4: chạy baseline bằng runtime hiện có (đảm bảo "chạy được").
- Buổi tối 5–6 và/hoặc 12–13: nếu latency chưa đạt, ưu tiên **TensorRT** hoặc **ORT + TensorRT EP**; dùng **trtexec** để benchmark/generate engine nhanh không cần viết app mới (TensorRT xác nhận trtexec dùng để benchmark và tạo serialized engine). [19]

Audio interface / microphone

Phương án	Ưu điểm	Nhược điểm	Khi nên chọn	"Test nhanh" gợi ý
USB mic (UAC) cắm thẳng Jetson	Setup nhanh; thường "cắm là chạy"; dễ thay thế	Chất lượng tuỳ mic; dễ nhiễu từ motor/USB	Bạn ưu tiên tiến độ, demo nhanh	`arecord -l` liệt kê capture devices [20]; ghi thử 10s, nghe lại
USB sound card + mic rời (TRRS/ 3.5mm)	Dễ gắn mic rời; có thể chọn mic tốt/ dài dây	Thêm 1 thiết bị; chất lượng phụ thuộc sound card	Khi muốn mic đặt xa Jetson/motor	Ghi/so sánh SNR trong 3 vị trí: gần miệng, trên thân robot, ngoài vỏ
Digital mic qua I2S (MEMS)	Tiềm năng nhiễu thấp hơn analog, gọn	Trên Jetson Nano, I2S có trên M.2 Key-E (cần tháo module để tới J18); tích hợp cơ khí/ driver phức tạp [21]	Khi cần thiết kế "product-like", mic nằm trong vỏ gọn	Prototype trước bằng USB; I2S chỉ nên làm khi USB "không cứu được"

Ghi chú: Jetson Nano DK có M.2 Key E connector (J18) bao gồm cả I2S theo user guide. [21]

Giải pháp nguồn pin độc lập

Phương án	Tóm tắt	Ưu điểm	Nhược điểm	Khi nên chọn	Ràng buộc theo tài liệu NVIDIA
Power bank → Micro-USB (J28)	Dùng power bank 5V cấp vào Micro-USB	Nhanh, rẻ, ít hàn nối	Có thể bị giới hạn dòng; dễ sụt áp; khó chịu tải ngoại vi/motor; Jetson cần tối thiểu 4.75V [1]	Demo rất nhẹ, ít USB, chưa gắn nhiều ngoại vi	Micro-USB mode 5V⎓2A và yêu cầu 4.75V min [1]
Pack pin (2S/ 3S Li-ion) → buck 5V → jack DC J25	Pin qua DC-DC buck ổn định 5V, cấp vào J25, bật bằng J48	Ổn định hơn, chịu tải tốt; phù hợp khi dùng 10W mode	Cần buck tốt (dòng lớn), dây/ đầu jack chuẩn 2.1×5.5mm; cần đi dây gọn	Khi robot chạy motor + Jetson cần ổn định	J25 là 5V⎓4A, J48 chọn nguồn; plug spec và khuyến nghị vượt 2A chuyển sang J25 [1]
Pack pin → buck 5V → cấp qua header J41	Cấp 5V qua 2 pin 5V (2.5A/ pin)	Lắp gọn trong vỏ, không cần jack ngoài	Rủi ro đấu nhầm; cần bảo vệ (fuse)	Khi bạn tự tin phần điện và muốn set-up "embedded"	NVIDIA nêu có thể cấp 5V⎓5A qua J41 (2.5A mỗi pin) [1]

Lộ trình hai tuần buổi tối với deliverables và acceptance criteria

Timeline dạng Mermaid Gantt cho mười bốn buổi tối

```
gantt
  title Roadmap hai tuần (buổi tối) cho Mone
  dateFormat  YYYY-MM-DD
  axisFormat  %d/%m

  section Nền tảng Audio + Jetson
  Mic/device discovery + chuẩn hóa ghi âm      :a1, 2026-03-28, 1d
  Baseline pipeline trên Jetson (offline)      :a2, 2026-03-29, 1d
  Live capture + đo latency + tegrastats       :a3, 2026-03-30, 1d
  Quản lý nhiệt sơ bộ + power mode + ổn định   :a4, 2026-03-31, 1d

  section Persona + prompt + hành vi tối giản
  Persona spec + response rules                :b1, 2026-04-01, 1d
  Prompt + safety/fallback + logging           :b2, 2026-04-02, 1d
  Tích hợp end-to-end (speech->emotion->reply) :b3, 2026-04-03, 1d

  section ESP32 + chuyển động + chassis
  ESP32 PWM + H-bridge bench test              :c1, 2026-04-04, 1d
  Chassis assembly + motor wiring              :c2, 2026-04-05, 1d
  Jetson<->ESP32 protocol + control loop       :c3, 2026-04-06, 1d
  Behavior mapping (emotion->motion)           :c4, 2026-04-07, 1d

  section Nguồn pin + nhiệt + tối ưu latency
  Pin độc lập + chống sụt áp                   :d1, 2026-04-08, 1d
  Quạt/pwm fan + enclosure airflow check       :d2, 2026-04-09, 1d
  Tối ưu inference (TensorRT/ORT) + latency    :d3, 2026-04-10, 1d
  Stress test + demo packaging                 :d4, 2026-04-11, 1d
```

> Ngày trên Gantt chỉ để "đánh dấu 14 buổi liên tiếp"; bạn có thể dịch lịch theo ngày thực tế của bạn.

Kế hoạch chi tiết từng buổi tối

Mỗi buổi tối dưới đây được thiết kế để hoàn thành trong **3–4 giờ**. Mỗi buổi đều có: (1) bước làm cụ thể, (2) deliverables, (3) acceptance criteria, (4) dụng cụ/phần mềm, (5) test cases, (6) checklist xử lý sự cố nhanh, (7) rủi ro & giảm thiểu, (8) backlog ban ngày.

Buổi tối đầu tiên: Mic stabilization "có số đo"

Mục tiêu: Jetson/Laptop (tuỳ bạn đang test ở đâu) ghi âm ổn định, format chuẩn hóa, và có bộ test nhanh lặp lại.

Các bước (3–4 giờ):
- Thiết bị & driver (30–45'): liệt kê sound card/device và chọn đúng input. - Dùng `arecord` vì đây là công cụ recorder dòng lệnh cho ALSA và hỗ trợ nhiều soundcards/devices. [7]
- Chuẩn hoá format (45'): chốt sample rate (ví dụ 16k), mono, S16_LE; ghi 10 file mẫu theo 3 khoảng cách.
- Bộ test "lặp lại" (60'): viết script chạy 20 lần (hoặc 2×10) để ghi âm + chạy infer trên file audio (tạm thời offline) và log: predicted label + confidence + thời gian.
- Review (30–45'): thống kê tỉ lệ ổn định và lỗi.

Deliverables:
- `audio_samples/` gồm file WAV theo naming chuẩn.
- `mic_smoke_test.csv` có cột: timestamp, device, rms/dbfs, label, confidence, t_record, t_infer.

Acceptance criteria:
- Ghi âm không bị "đứt/gãy" hoặc silent bất thường.
- Cùng một câu + điều kiện môi trường tương tự: dự đoán không nhảy loạn (đề xuất: ≥80% ổn định trong 20 lần).

Dụng cụ/phần mềm:
- Jetson hoặc laptop Linux; `alsa-utils` (arecord/aplay). [20]

Test cases tối thiểu:
- Case A: phòng yên, nói gần mic.
- Case B: bật quạt/tiếng nền.
- Case C: đặt mic gần hệ motor (chưa chạy) để kiểm nhiễu nền.

Troubleshooting nhanh (audio):
- Không thấy mic: chạy `arecord -l` để liệt kê capture devices. [20]
- Ghi được nhưng lỗi format: chạy `arecord -v ...` để xem thiết bị hỗ trợ (ALSA có thể không chấp nhận một số hw params; lúc đó dùng "plug" device hoặc chỉnh rate/format). [22]

Rủi ro & giảm thiểu:
- Rủi ro: PulseAudio can thiệp gây trễ/giật. Giảm thiểu: ưu tiên ALSA trực tiếp bằng arecord; nếu cần PulseAudio, tham khảo cơ chế latency control. [8]

Backlog ban ngày (ưu tiên):
- Nếu mic kém: đặt mua/đổi mic (xem bảng mic options).
- Chuẩn bị dây nối dài/clip để cố định mic đúng vị trí.

Snippet lệnh gợi ý (bạn thay card/device theo máy):

```
# Liệt kê thiết bị capture
arecord -l

# Ghi 10s, 16kHz, mono, 16-bit little endian
arecord -D plughw:1,0 -f S16_LE -r 16000 -c 1 -d 10 sample.wav
```

```
# Nghe lại
aplay sample.wav
```

[23]

Buổi tối thứ hai: Baseline inference chạy trên Jetson (offline)

Mục tiêu: chạy được model emotion trên Jetson (chưa cần live mic), tái lập môi trường.

Các bước:
- Power sanity (10'): đảm bảo nguồn đủ; nếu có nhiều USB/peripherals, cân nhắc chuyển sang nguồn mạnh hơn theo hướng dẫn NVIDIA (J25 5V⎓4A qua J48 hoặc J41). [1]
- Setup môi trường (90'): tạo venv/conda, cài dependencies tối thiểu; copy model + code.
- "Offline infer" (60'): chạy infer trên 10 file WAV đã thu buổi 1.
- Log & timing (30–45'): thêm timer từng bước (load model, preprocess, infer).

Deliverables:
- `run_infer_offline.sh` hoặc `python infer_offline.py --input_dir audio_samples/`
- `jetson_env_freeze.txt` (pip freeze)

Acceptance criteria:
- Chạy hết batch file WAV không crash; thời gian infer ổn định (dao động nhỏ).

Dụng cụ/phần mềm:
- Jetson Nano + microSD; NVIDIA mô tả tối thiểu microSD 16GB UHS-1 để setup dev kit. [3]
- Tegrastats để quan sát tài nguyên (chuẩn bị dùng từ buổi 3). [24]

Rủi ro:
- Thiếu RAM/space trên microSD → giảm thiểu: dọn image, tắt GUI khi không cần (đưa vào backlog), hoặc chuyển project nhẹ.

Backlog ban ngày:
- Nếu microSD chậm: đổi microSD nhanh hơn (UHS-1/V30).

Buổi tối thứ ba: Live capture trên Jetson + đo latency chuẩn hoá + tegrastats

Mục tiêu: Jetson "nghe" mic thật (live) và xuất prediction + đo latency end-to-end (chưa motor).

Các bước:
- Live capture (45–60'): dùng arecord capture trực tiếp trên Jetson; hoặc dùng Python audio nếu cài kịp.
- Đo latency pipeline (60'): chuẩn hóa timer:
- `t0` start record
- `t1` audio ready
- `t2` preprocess done

- `t3` infer done
- `t4` response chosen
- Giám sát tài nguyên (30'): chạy **tegrastats** (tiện ích báo cáo memory + processor) để xem CPU/GPU/thermal trong lúc chạy test. [2]
- Báo cáo nhanh (30–45'): xuất CSV latency.

Deliverables:
- `latency_log.csv` (ít nhất 30 mẫu)
- `tegrastats.log` (khoảng 5–10 phút)

Acceptance criteria:
- Live capture không drop sample; pipeline chạy 30 lần liên tục.
- Có P50/P95 latency baseline để so sau này.

Tools:
- tegrastats: có hướng dẫn stop/kill và xác nhận preinstalled trong Jetson Linux docs. [25]

Rủi ro & giảm thiểu:
- Nếu Jetson nóng: chuẩn bị buổi 4 gắn quạt vào J15 (PWM+tach). [5]

Buổi tối thứ tư: Power mode + thermal "đủ để chạy lâu" + ổn định I/O

Mục tiêu: giảm crash do nguồn/nhiệt; chốt "profile chạy ổn" để chuyển sang ESP32/chassis.

Các bước:
- Kiểm tra/cấu hình nguồn:
- Nếu dùng micro-USB 2A mà gắn nhiều ngoại vi, chuyển sang J25 5V⎓4A bằng jumper J48 (NVIDIA mô tả rõ J25, J28, J48 và logic chọn nguồn). [1]
- Thermals: gắn quạt 4-pin vào **fan header J15** (hỗ trợ PWM & tach). [5]
- Lưu ý chọn quạt phù hợp điện áp; ví dụ Noctua nhấn mạnh bản 12V PWM không hoạt động, cần bản 5V PWM cho Jetson Nano. [26]
- Chạy soak test 30–45 phút: pipeline live + tegrastats; ghi max temp/công suất.
- Chốt "operating mode" theo power budget: NVIDIA mô tả module hỗ trợ 2 power modes (10W default và 5W), giới hạn CPU/GPU theo budget. [1]

Deliverables:
- `stability_report.md`: nguồn đang dùng, quạt, max temp, lỗi gặp.

Acceptance criteria:
- Chạy liên tục 30–45 phút không reset; tegrastats không báo tình trạng bất thường.

Backlog ban ngày:
- In/chuẩn bị bracket gắn quạt, luồng gió trong vỏ.

Buổi tối thứ năm: Persona spec + response policy (không cần LLM là vẫn demo được)

Mục tiêu: Mone có tính cách nhất quán, phản hồi theo cảm xúc và có fallback khi "không chắc".

Các bước:
- Viết persona 1 trang: xưng hô, tone, nguyên tắc trả lời ngắn.
- Tạo bảng rule: `emotion → intent → response template`.
- Cài logging: lưu input label/confidence, response chosen.

Deliverables:
- `persona.md`
- `response_rules.yaml/json`

Acceptance criteria:
- 3 tình huống (vui/buồn/không rõ) đều ra câu phù hợp và nhất quán.

Backlog ban ngày:
- Soạn thêm 50 câu trả lời theo persona để chống lặp.

Buổi tối thứ sáu: Prompt + safety/fallback + chuẩn bị "API" nội bộ

Mục tiêu: nếu bạn dùng LLM/Chat API thì prompt sẵn sàng; nếu không, prompt vẫn là "spec" để mở rộng.

Các bước:
- Viết prompt hệ thống + prompt ngữ cảnh.
- Fallback: nếu confidence < ngưỡng, hỏi lại hoặc phản hồi trung tính.
- Chuẩn hóa output schema: `{"emotion":..., "confidence":..., "reply":..., "action":...}`

Deliverables:
- `prompt_system.txt` + `prompt_examples.md`
- `schema.json` + validator đơn giản

Acceptance criteria:
- Chạy 20 lần live: không có response "lạc persona".

Buổi tối thứ bảy: End-to-end social loop (nghe → hiểu → trả lời) + logging

Mục tiêu: đường ống social hoàn chỉnh (chưa motor) trên Jetson.

Các bước:
- Ghép live capture + emotion + response policy/prompt.
- Tạo "one-command run": `./run_mone.sh`.
- Ghi 1 video/ghi màn hình (chỉ cần làm bằng chứng).

Deliverables:
- `run_mone.sh`
- `demo_social_loop.mp4` (hoặc log)

Acceptance criteria:
- 10 phút chạy liên tục không crash; log có đủ timestamp cho latency.

Buổi tối thứ tám: ESP32 PWM + H-bridge bench test (chưa gắn chassis)

Mục tiêu: ESP32 điều khiển được motor bằng PWM, có stop và giới hạn duty.

Các bước:
- Chọn LEDC hoặc MCPWM:
- LEDC dễ cho PWM tổng quát (ESP-IDF nêu LEDC có thể dùng tạo PWM cho mục đích khác). [9]
- MCPWM phù hợp cho motor control; tài liệu có phần "simple configuration" brushed DC + H-bridge. [27]
- Bench test 1 motor trước, sau đó 2 motor.
- Thêm "failsafe": nếu không nhận lệnh trong X ms → stop.

Deliverables:
- Firmware `esp32_motor_v1/`
- `protocol.md` (format lệnh)

Acceptance criteria:
- Motor chạy 5 lệnh cơ bản ổn định; stop luôn hoạt động.

ESP-IDF snippet minh hoạ (LEDC PWM):

```
// LEDC có thể dùng để tạo PWM cho mục đích khác ngoài LED. [9]
ledc_timer_config_t timer = {
  .speed_mode = LEDC_LOW_SPEED_MODE,
  .timer_num = LEDC_TIMER_0,
  .duty_resolution = LEDC_TIMER_10_BIT,
  .freq_hz = 20000,
  .clk_cfg = LEDC_AUTO_CLK
};
ledc_timer_config(&timer);

ledc_channel_config_t ch = {
  .gpio_num = MOTOR_PWM_GPIO,
  .speed_mode = LEDC_LOW_SPEED_MODE,
  .channel = LEDC_CHANNEL_0,
  .timer_sel = LEDC_TIMER_0,
  .duty = 0
};
ledc_channel_config(&ch);
```

```
// set duty
ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0, duty);
ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0);
```

Backlog ban ngày:
- Nếu H-bridge nóng: đổi driver phù hợp dòng motor; chuẩn bị heatsink.

Buổi tối thứ chín: Chassis assembly + wiring sạch

Mục tiêu: gắn motor lên khung gầm, dây đi gọn, test không chạm chập.

Các bước:
- Lắp cơ khí khung; cố định H-bridge và ESP32.
- Đi dây có "strain relief"; tách dây motor (dòng lớn) khỏi dây audio nếu có thể.
- Test chạy bánh xe trên giá đỡ (không đặt xuống đất) trước.

Deliverables:
- `wiring_map.png` (vẽ tay cũng được) + ảnh thực tế.

Acceptance criteria:
- Chạy bánh trái/phải độc lập; không sụt áp làm reset ESP32.

Buổi tối thứ mười: Jetson ↔ ESP32 protocol + control loop

Mục tiêu: Jetson gửi lệnh, ESP32 nhận và điều khiển motor theo protocol ổn định.

Các bước:
- Chọn đường truyền: UART/USB serial (dễ nhất) hoặc Wi-Fi.
- Implement sender trên Jetson (Python): gửi lệnh `STOP/FWD/BACK/LEFT/RIGHT` + speed.
- Implement ack/heartbeat: ESP32 gửi `OK` hoặc trạng thái.

Deliverables:
- `jetson_motor_client.py`
- `esp32_motor_v2` (có protocol)

Acceptance criteria:
- 100 lệnh liên tục không mất sync; stop luôn ưu tiên cao nhất.

Buổi tối thứ mười một: Behavior mapping (emotion → motion) + safety

Mục tiêu: robot "social" không chỉ nói mà còn có hành vi đơn giản, an toàn.

Các bước:
- Mapping gợi ý (tối giản):
- vui → tiến nhẹ 0.5s rồi dừng
- buồn → tiến chậm 0.3s (hoặc nghiêng/đổi hướng nếu có)
- không rõ → đứng yên
- Thêm "safety envelope": tốc độ max, thời gian max mỗi hành động, kill-switch.

Deliverables:
- `behavior.yaml` + unit test nhỏ: emotion giả → action.

Acceptance criteria:
- 20 lần chạy mapping: không có hành vi "giật" hoặc kéo dài ngoài dự kiến.

Buổi tối thứ mười hai: Pin độc lập + chống brownout (ưu tiên ổn định Jetson)

Mục tiêu: robot chạy không dây; Jetson không reset khi motor hoạt động.

Các bước:
- Chọn phương án nguồn theo bảng nguồn phía trên, bám đúng ràng buộc NVIDIA:
- Nếu tải cao: dùng J25 5V⎓4A (bật J48) hoặc cấp qua J41 5V⎓5A (2.5A/pin). [1]
- Tách nguồn logic/motor (khuyến nghị): ít nhất tách buck/rail; nhưng **chung ground** để tín hiệu điều khiển đúng.
- Test "worst case": đứng yên → tăng tốc → đổi hướng → stop, lặp 20 lần.

Deliverables:
- `power_notes.md`: sơ đồ nguồn, thông số buck, cầu chì/công tắc (nếu có).

Acceptance criteria:
- Jetson chạy pipeline + motor đồng thời 10 phút không reset; điện áp không tụt dưới ngưỡng gây lỗi (NVIDIA nêu module cần tối thiểu 4.75V). [1]

Backlog ban ngày:
- Làm hộp pin/đế pin chắc chắn; sạc và kiểm tra BMS (an toàn Li-ion).

Buổi tối thứ mười ba: Thermal + airflow + "đóng vỏ" bản nháp

Mục tiêu: không quá nhiệt khi đóng vỏ; quạt chạy đúng.

Các bước:
- Gắn quạt vào J15 (PWM+tach). [5]
- Chạy soak test 20–30 phút trong trạng thái gần "đóng vỏ".
- Ghi tegrastats trước/sau khi đóng vỏ. [24]

Deliverables:
- `thermal_before_after.csv` + ghi chú thay đổi cần làm cho vỏ.

Acceptance criteria:
- Nhiệt độ ổn định, không tăng không kiểm soát; nếu tăng, phải có giải pháp (khoét gió, đổi hướng quạt, giảm power mode).

Buổi tối thứ mười bốn: Tối ưu latency <2s + stress test + demo packaging

Mục tiêu: chạm/tiệm cận KPI latency và đóng gói demo.

Các bước:
- Chốt phép đo latency (đúng timer buổi 3) và đo lại sau tối ưu.
- Nếu inference là nút thắt:
- Dùng TensorRT/ORT+TensorRT EP; benchmark/engine bằng **trtexec** (TensorRT xác nhận trtexec dùng để benchmark và tạo serialized engines). [19]
- Nếu audio là nút thắt: giảm window record, dùng VAD, tránh buffer quá dài.
- Stress test 60–120 phút: chạy liên tục, log lỗi, tegrastats. [2]
- Demo packaging: auto-run script, README "1 trang".

Deliverables:
- `latency_report_final.md` (P50/P95, trước/sau)
- `demo_runbook.md` (cách bật, kịch bản demo, xử lý sự cố)
- 1 video demo hoàn chỉnh

Acceptance criteria:
- Latency P95 đạt hoặc tiệm cận <2 giây theo phép đo thống nhất; demo chạy ổn định.

Chiến lược test, troubleshooting nhanh, và quản trị rủi ro

Bộ test cases tối thiểu (khuyến nghị duy trì xuyên suốt)

Audio & emotion inference - A1: Nói 10 câu vui, 10 câu buồn trong phòng yên (+ log confidence).
- A2: Lặp lại A1 với noise nền (quạt).
- A3: Lặp lại A1 khi motor "đang quay không tải" (để phát hiện nhiễu điện từ/nguồn).

Motor & safety - M1: 100 lệnh ngẫu nhiên (seed cố định) → không mất điều khiển.
- M2: Mất kết nối Jetson 10s → ESP32 phải stop (failsafe).
- M3: Kill-switch (phần mềm) hoạt động ngay.

Power/thermal - P1: Sequence "tệ nhất": forward 1s → left 0.5s → back 1s → right 0.5s (×20).
- T1: Soak 30 phút vỏ mở; T2: Soak 30 phút vỏ gần đóng; so sánh bằng tegrastats. [24]

Checklist xử lý sự cố nhanh (theo ưu tiên tích hợp)

Jetson không ổn định / reset - Kiểm nguồn: Micro-USB 5V⎓2A dễ thiếu khi tăng tải; NVIDIA khuyến nghị nếu tải vượt 2A thì chuyển sang J25 5V⎓4A (bằng J48) hoặc cấp qua J41 5V⎓5A; đảm bảo không tụt dưới 4.75V. [1]
- Giảm ngoại vi USB, hoặc chuyển sang nguồn mạnh.

Mic không thu / thu sai device - `arecord -l` để kiểm tra capture devices; arecord là tool recorder cho ALSA. [7]
- Dùng `arecord -v` xem thiết lập thật sự; nếu hw không hỗ trợ format, đổi rate/format hoặc dùng plug device. [22]

Latency cao bất thường - Phân rã latency: record vs preprocess vs infer vs response.
- Dùng tegrastats để xem CPU/GPU/memory/thermal khi chạy. [2]
- Nếu inference bottleneck: dùng TensorRT (SDK tối ưu inference) hoặc ORT + TensorRT EP; benchmark bằng trtexec. [28]

Motor rung/giật - PWM freq quá thấp → tăng lên (ví dụ 20kHz) và ramp tốc độ.
- Nếu dùng ESP-IDF: LEDC dùng được để tạo PWM tổng quát. [9]
- Nếu cần bài bản motor: MCPWM có mô hình cho motor control và ví dụ brushed DC + H-bridge. [29]

Overheat - Jetson DK có fan header J15 PWM+tach; passive heatsink hỗ trợ 10W ở 25°C, nếu cần phải thêm cooling. [5]
- Chọn quạt đúng điện áp (ví dụ 5V PWM). [26]

Risk mitigation theo từng "cụm" (để hợp lịch buổi tối)

- **Rủi ro phụ thuộc phần cứng đến trễ:** luôn có "fallback tối nay" (ví dụ nếu chưa lắp chassis thì vẫn làm protocol + simulator).
- **Rủi ro convert TensorRT thất bại do op không hỗ trợ:** fallback ORT + CUDA EP hoặc chạy baseline runtime; ORT docs khuyến nghị đăng ký cả TensorRT EP và CUDA EP để fallback các node TensorRT không hỗ trợ. [13]
- **Rủi ro nguồn sụt do motor:** theo đúng khả năng cấp nguồn Jetson (J25/J41) và tách rail motor khi có thể. [1]

Backlog ưu tiên cho ban ngày và các bước sau hai tuần

Backlog ban ngày ưu tiên cao (vì buổi tối không đủ thời gian chờ)

- In 3D vỏ/khung/giá quạt (thời gian in dài).
- Mua/đổi mic theo bảng lựa chọn; chuẩn bị dây chống nhiễu.
- Đóng gói pin: hộp pin, công tắc, cầu chì, sạc/kiểm BMS (an toàn).
- Chuẩn bị "bàn test" cho chassis (giá đỡ bánh xe, dụng cụ đo điện áp).
- Nếu cần tối ưu TensorRT: chạy convert/benchmark dài (trtexec) nhiều lần để tìm cấu hình tốt (FP16/INT8 nếu phù hợp) — trtexec được NVIDIA nêu rõ có thể dùng để benchmark và tạo engine. [19]

Backlog sau hai tuần (khi demo đã chạy)

- Nâng cấp audio: VAD tốt hơn, noise suppression, mic array.
- Hành vi social phong phú: micro-gestures, “approach/avoid”, điều hướng an toàn.
- Monitoring dashboard: xuất metrics từ tegrastats/log để nhìn nhiệt/latency theo thời gian (tegrastats là nguồn dữ liệu chuẩn). [24]

Link tham khảo nhanh (đặt trong code theo yêu cầu hiển thị URL)

```
NVIDIA Jetson Nano Developer Kit User Guide (PDF):
https://developer.download.nvidia.com/embedded/L4T/r32-3-1_Release_v1.0/
Jetson_Nano_Developer_Kit_User_Guide.pdf

NVIDIA Jetson Linux Developer Guide – tegrastats:
https://docs.nvidia.com/jetson/archives/r36.4.3/DeveloperGuide/AT/
JetsonLinuxDevelopmentTools/TegrastatsUtility.html

NVIDIA TensorRT Documentation + trtexec:
https://docs.nvidia.com/deeplearning/tensorrt/latest/
https://docs.nvidia.com/deeplearning/tensorrt/latest/reference/command-line-
programs.html

ONNX Runtime – TensorRT Execution Provider:
https://onnxruntime.ai/docs/execution-providers/TensorRT-ExecutionProvider.html

ESP-IDF – LEDC (PWM):
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/
peripherals/ledc.html

ESP-IDF – MCPWM (motor control, H-bridge example):
https://docs.espressif.com/projects/esp-idf/en/release-v4.4/esp32/api-reference/
peripherals/mcpwm.html

ALSA arecord/aplay manpage:
https://manpages.debian.org/trixie/alsa-utils/arecord.1.en.html
```

---

[1] [3] [5] [21] Jetson Nano Developer Kit
https://developer.download.nvidia.com/embedded/L4T/r32-3-1_Release_v1.0/Jetson_Nano_Developer_Kit_User_Guide.pdf

[2] [24] Tegrastats Utility - NVIDIA Documentation Hub
https://docs.nvidia.com/jetson/archives/r34.1/DeveloperGuide/text/AT/JetsonLinuxDevelopmentTools/TegrastatsUtility.html?utm_source=chatgpt.com

[4] [9] LED Control (LEDC) - ESP32 - — ESP-IDF Programming Guide v6.0 documentation
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/ledc.html?utm_source=chatgpt.com

6 11 28 NVIDIA TensorRT Documentation
https://docs.nvidia.com/deeplearning/tensorrt/latest/?utm_source=chatgpt.com

7 20 22 23 arecord (1) — alsa-utils — Debian trixie — Debian Manpages
https://manpages.debian.org/trixie/alsa-utils/arecord.1.en.html?utm_source=chatgpt.com

8 Latency Control – Developer Documentation – PulseAudio
https://www.freedesktop.org/wiki/Software/PulseAudio/Documentation/Developer/Clients/LatencyControl/?utm_source=chatgpt.com

10 Motor Control Pulse Width Modulator (MCPWM) - ESP32 - — ESP-IDF ...
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/mcpwm.html?utm_source=chatgpt.com

12 Execution Providers | onnxruntime
https://onnxruntime.ai/docs/execution-providers/?utm_source=chatgpt.com

13 TensorRT Execution Provider - ONNXRuntime
https://iot-robotics.github.io/ONNXRuntime/docs/execution-providers/TensorRT-ExecutionProvider.html?utm_source=chatgpt.com

14 NVIDIA - TensorRT | onnxruntime
https://onnxruntime.ai/docs/execution-providers/TensorRT-ExecutionProvider.html?utm_source=chatgpt.com

15 tf.lite.Interpreter | TensorFlow v2.16.1
https://www.tensorflow.org/api_docs/python/tf/lite/Interpreter?utm_source=chatgpt.com

16 18 LiteRT Delegates | Google AI Edge | Google AI for Developers - TensorFlow
https://ai.google.dev/edge/litert/performance/delegates?utm_source=chatgpt.com

17 TFLite on GPU - Google Open Source
https://android.googlesource.com/platform/external/tensorflow/%2B/refs/heads/main/tensorflow/lite/delegates/gpu/README.md?utm_source=chatgpt.com

19 Command-Line Programs — NVIDIA TensorRT
https://docs.nvidia.com/deeplearning/tensorrt/latest/reference/command-line-programs.html?utm_source=chatgpt.com

25 Tegrastats Utility — NVIDIA Jetson Linux Developer Guide 1 documentation
https://docs.nvidia.com/jetson/archives/r36.2/DeveloperGuide/AT/JetsonLinuxDevelopmentTools/TegrastatsUtility.html?utm_source=chatgpt.com

26 Which is the best fan for the NVIDIA Jetson Nano? - Noctua
https://noctua.at/en/support/faqs/which-is-the-best-fan-for-the-nvidia-jetson-nano?utm_source=chatgpt.com

27 29 Motor Control Pulse Width Modulator (MCPWM) - ESP32 - — ESP-IDF ...
https://docs.espressif.com/projects/esp-idf/en/release-v4.4/esp32/api-reference/peripherals/mcpwm.html?utm_source=chatgpt.com